

Software Engineering at Google - Resumen

Jake and Drosh

24-06-2025

Testing Overview, Cap. 11

Los aspectos más fundamentales del Capítulo 11 del libro *Software Engineering at Google*, titulado “**Prueba general**”, pueden resumirse de la siguiente manera:

1. Finalidad de las pruebas automatizadas

- Las pruebas automatizadas tienen como objetivo **detectar errores lo antes posible** y **dar confianza** sobre el comportamiento del sistema.
 - Una prueba efectiva debe tener: una entrada definida, una salida esperada, una función específica a evaluar y un entorno controlado.
 - Cuanto **más temprano se detecta un error**, **más barato y rápido es corregirlo** (principio de “shifting left”).
-

2. Beneficios clave

- **Facilitan cambios seguros** en el código.
 - Mejoran el **diseño de APIs** al forzar que el código sea más modular y testeable.
 - Incrementan la **productividad** del desarrollador.
 - Hacen que las **revisiones de código** sean más efectivas al mostrar evidencia automática del correcto funcionamiento.
-

3. Ciclo de pruebas automatizadas

- **Escribir pruebas:** Códigos que ejercen partes específicas del sistema.
 - **Ejecutar pruebas:** De forma frecuente, confiando en la constancia de las máquinas.
 - **Reaccionar ante fallos:** Resolver errores inmediatamente al detectarlos.
-

4. Clasificación por tamaño

- **Pruebas pequeñas:** Rápidas, deterministas, sin uso de red ni múltiples procesos.
 - **Pruebas medianas:** Permiten concurrencia limitada y uso de `localhost`.
 - **Pruebas grandes:** Involucran múltiples máquinas y procesos distribuidos.
 - **Pruebas herméticas:** Independientes del entorno externo, más confiables.
-

5. Clasificación por alcance

- **Pruebas unitarias:** Validan funciones o métodos específicos.
- **Pruebas de integración:** Evalúan la interacción entre componentes.
- **Pruebas de sistema o extremo a extremo:** Prueban el comportamiento del sistema completo.

Pirámide de pruebas sugerida:

- 80 % pruebas unitarias
 - 15 % de integración
 - 5 % de extremo a extremo
-

6. Buenas prácticas y filosofía

- “**Regla de Beyoncé**”: Si algo es importante, debe tener una prueba.
 - **Cobertura de código**: Es útil, pero no suficiente. Importa **qué comportamientos se están probando**, no solo qué líneas se ejecutan.
 - **Pruebas frágiles o inestables**: Afectan la confianza y deben evitarse. Los conjuntos de pruebas deben ser rápidos, confiables y deterministas.
-

7. Escalabilidad y cultura

- Google trabaja en un **monorepo gigantesco**, sin ramificaciones; todo se integra a la cabeza del repositorio.
 - Usa una **infraestructura de CI** robusta (como la plataforma TAP) para correr pruebas constantemente.
 - Tiene una **cultura fuerte de pruebas**, promovida desde la incorporación de los ingenieros:
 - **Testing Grouplet** y programas como *Testing on the Toilet* y *Test Certified* fomentaron prácticas sostenidas.
 - La herramienta **Project Health** hoy proporciona métricas de calidad en tiempo real.
-

8. Límites de las pruebas automatizadas

- No reemplazan por completo a las pruebas manuales.
 - Deben centrarse en lo predecible, permitiendo que los testers humanos se enfoquen en exploración y análisis cualitativo.
-

Conclusión

Este capítulo sienta las bases del enfoque de Google hacia el testing: **automatización disciplinada, detección temprana, clasificación estratégica de pruebas y una cultura organizacional que prioriza la calidad desde el primer día.**

Unit testing, Cap. 12

Los aspectos más fundamentales del Capítulo 12 de *Software Engineering at Google*, titulado “**Unit Testing**”, se pueden resumir en los siguientes puntos clave:

1. ¿Qué son las pruebas unitarias?

- Son **pruebas de alcance reducido** que verifican una pequeña unidad del código, como un método o clase.
 - Suelen ser de **tamaño pequeño**: rápidas, deterministas y se ejecutan en un único proceso.
-

2. ¿Por qué son importantes?

- Son la base de la **productividad de los desarrolladores**.
 - Permiten:
 - Detección rápida de errores.
 - Cambios seguros en el código.
 - Alta cobertura de prueba.
 - Diagnóstico simple al fallar.
 - Documentación viva y ejemplos claros de uso.
-

3. Mantenibilidad: el valor a largo plazo

Una buena prueba unitaria debe ser **estable, clara y fácil de mantener**.

Problemas comunes:

- **Pruebas frágiles**: fallan por cambios menores en la implementación, aunque no haya errores reales.

- **Pruebas poco claras:** difíciles de leer o entender, lo que complica el diagnóstico.

Prevención:

- **Probar a través de APIs públicas**, no de detalles internos.
 - **Probar el estado final**, no las interacciones internas.
 - Evitar uso excesivo de mocks; preferir objetos reales simples.
-

4. Pruebas claras y bien escritas

Una prueba clara:

- Tiene toda la información necesaria (**completa**).
 - Evita lo innecesario (**concisa**).
 - Usa el formato **Given-When-Then** para expresar comportamientos.
 - **Nombra** cada prueba de forma descriptiva.
 - **Evita lógica compleja** (if, for, etc.) dentro de las pruebas.
 - **Incluye buenos mensajes de error**, que informen qué falló y por qué.
-

5. Código compartido en pruebas: DAMP, no DRY

- Se favorece **DAMP** (*Descriptive And Meaningful Phrases*) sobre DRY (*Don't Repeat Yourself*), priorizando claridad sobre reutilización.
- Recomendaciones:
 - Usar **métodos auxiliares** con valores por defecto para construir datos de prueba.
 - Usar **configuración compartida** cuando no afecte la comprensión.

- Reutilizar código **sólo si mejora la legibilidad** y se justifica su complejidad.
 - La **infraestructura de prueba** (reutilizada entre suites) debe tratarse como código de producción, con estándares y pruebas propias.
-

6. Conclusión

Las pruebas unitarias son **una herramienta poderosa y esencial para la calidad del software**, pero deben diseñarse con cuidado para que sean:

- Mantenibles en el tiempo.
- Útiles para detectar errores reales.
- Claras para quien las escribe y quien las lee.

Mal utilizadas, pueden generar más costo que beneficio. Bien hechas, son una de las herramientas más valiosas para desarrollar software confiable a escala.